Lesson 3: Document Loading and Chunking (Day 30)

> ***Goal:*** *In Lesson 2, we learned how to talk to LLMs. But an LLM doesn't know about private company data (leave policies, customer records, manuals). Before we can search documents, we must* ***read them from disk*** *and* ***slice them into digestible pieces called chunks****. Today we learn the* ***12 different ways to cut text*** *and how to choose the right one.*

□ The Project We Are Building

A company has an internal folder of policies in `company_docs/`:

- `leave_policy.txt` (vacation and sick leave rules)
- `working_hours.txt` (office timings, lunch breaks, overtime)
- `laptop_rules.txt` (IT security and equipment rules)
- `expense_policy.txt` (reimbursement limits)

We write a Python program that reads all files and splits them into clean chunks ready for vector storage:

```
company_docs/
  ├── leave_policy.txt      ──►  leave_policy.txt-0: "Every employee gets 20 days of paid leave..."
  │                              leave_policy.txt-1: "To take leave, send a request 5 days before..."
  │                              leave_policy.txt-2: "Sick leave is separate from paid leave..."
  ├── working_hours.txt     ──►  working_hours.txt-0: "The office is open from 9 am to 5 pm..."
  ├── laptop_rules.txt      ──►  laptop_rules.txt-0: "Every new employee gets one laptop..."
  └── expense_policy.txt    ──►  expense_policy.txt-0: "The hotel limit is Rs 15000 per night..."
```

□ Vocabulary: Terms You Must Know

Word	What it Means	Everyday Real-World Analogy
Document	A single raw file containing text (e.g., `leave_policy.txt`, a PDF, or a web page).	An entire chapter of a textbook.
Load	Reading the contents of a file off your hard drive into Python memory as a string.	Opening a physical book and looking at the page.
Chunk	A small, focused slice of text cut out of a document.	A single paragraph or note card cut out of that chapter.
Chunking	The algorithmic strategy used to split a big document into small chunks.	Slicing a loaf of bread into individual slices.
Overlap	Repeating the last few words or sentences of Chunk 1 at the beginning of Chunk 2.	Like overlapping roof tiles so rain doesn't fall through the cracks.
Metadata	Extra background facts saved alongside each chunk (e.g., source file name, heading, date, author).	The label on a library book spine telling you the author and shelf number.
Source	The original file path or name where the chunk came from.	Citing the bibliography in an academic paper.

□ Why Do We Need to Chunk? (Why Not Feed the Whole File?)

Why can't we just take the entire `leave_policy.txt` file (or a 200-page PDF) and turn it into one single vector?

1. Vector Blurring (The "Smoothie" Problem)

- An embedding vector is a single point in math space representing an **idea**.
- If a document talks about vacation days, sick leaves, maternity leave, and disciplinary rules all at once, its embedding vector becomes a **blurry average of all four topics**.
- When an employee asks: *"How many sick days do I get?"*, the question is razor-sharp. The blurry 50-topic document vector won't match it closely!

- **Analogy:** If you blend apples, bananas, spinach, and chocolate into a smoothie, it doesn't taste like an apple anymore.

2. Token Limits and Cost

- If you retrieve an entire 20-page document for every small question, you will send 10,000 tokens to the LLM on every query.
- That makes replies very slow and very expensive.

3. But Don't Make Chunks Too Small!

- If your chunk is just 4 words: *"It is 10 days."*
- What is 10 days? Sick leave? Notice period? Hotel stay? The chunk has lost all context and is useless.
- **The Golden Rule: One chunk = One complete idea that can be understood on its own.**

Step 1: Loading Documents with Python (`pathlib.Path`)

Modern Python uses the built-in `pathlib` module to handle files cleanly across Windows, Mac, and Linux:

```
from pathlib import Path

def load_documents(folder):
    docs = []
    # sorted(...) keeps files in alphabetical order
    # .glob("*.txt") finds all text files in that folder
    for path in sorted(Path(folder).glob("*.txt")):
        docs.append({
            "source": path.name,                                  # e.g. "leave_policy.txt"
            "text": path.read_text(encoding="utf-8")           # the full text inside
        })
    return docs

docs = load_documents("company_docs")
for doc in docs:
    print(f"{doc['source']:20} | {len(doc['text'].split())} words")
```

□ Why This Code?

- `Path("company_docs").glob("*.txt")`: Automatically searches the folder for every file ending in `.txt`.
- `path.read_text(encoding="utf-8")`: Opens, reads, and closes the file safely in one line, decoding special characters properly.

Step 2: The 12 Ways to Chunk Text

There is no single "magic" way to chunk. Here are the 12 industry strategies:

#	Strategy	How It Cuts	Cost	Best Used For
1	**Fixed-Size**	Every N characters or words, blindly	Free & Instant	Quick prototypes, log files
2	**Sliding Window (Overlap)**	Fixed-size, but chunks share 20–30% text	Very Cheap	Audio transcripts, chat histories
3	**Sentence-Based**	At every `.`, `?`, `!`	Very Cheap	Language translation, sentiment
4	**Paragraph-Based**	At every double newline `\n\n`	Very Cheap	Policies, clean articles, emails
5	**Recursive**	Tries paragraphs first; if too big, falls back to sentences	Very Cheap	**Industry Default** for general PDFs/docs
6	**Semantic**	Cuts where embeddings show topic shifted	Medium	High-accuracy legal/medical contracts
7	**Hierarchical**	Small child chunks for search, big parent for LLM	Medium	Long technical books, complex contracts
8	**Contextual**	Prepends document title/section to each chunk	Cheap	Enterprise knowledge bases
9	**Topic-Based**	Clusters sentences by subject (even if far apart)	Medium	Meeting minutes, rambling transcripts
10	**Modality-Specific**	Text by paragraphs, tables row-by-row with headers	Cheap/Medium	Financial reports, bank statements
11	**AI-Driven**	Prompts an LLM to choose where to split	Expensive	High-value, critical documents

#	Strategy	How It Cuts	Cost	Best Used For
12	Agentic	An autonomous AI agent picks different chunkers	Very Expensive	Multi-modal heterogeneous archives

Let's look at the implementation of the most important ones:

Strategy 1 & 2: Fixed-Size vs Sliding Window (Overlap)

```
def chunk_by_size(text, size=100, overlap=0):
    chunks = []
    step = size - overlap  # How far forward the window slides
    for start in range(0, len(text), step):
        chunks.append(text[start:start + size])
    return chunks

sample = "Every employee gets 20 days of paid leave. Unused leave does not carry over to next year."

# Fixed size without overlap:
print("No overlap:", chunk_by_size(sample, size=40, overlap=0))
# Notice words get sliced in half: ['Every employee gets 20 days of paid leav', 'e. Unused leave does not carry over to n', ...]

# With 15 characters overlap:
print("With overlap:", chunk_by_size(sample, size=40, overlap=15))
```

> ***Why overlap matters:*** *Overlap ensures that a sentence split across the boundary of Chunk 1 is preserved intact in Chunk 2.*

Strategy 3: Sentence-Based Chunking

Splits at sentence terminators (`. ! ?`) using regular expressions:

```
import re

def chunk_by_sentence(text):
    # Regex looks for punctuation followed by space or newline
    sentences = re.split(r"(?<=[.!?])\s+|\n\n", text.strip())
    return [s.strip() for s in sentences if s.strip()]
```

Strategy 4: Paragraph-Based Chunking (Our Course Default!)

Human authors naturally put **one thought per paragraph**. A blank line (`\n\n`) separates thoughts.

```
def chunk_by_paragraph(text):
    # Split on double newlines
    parts = [p.strip() for p in text.split("\n\n")]
    # Discard tiny fragments like single-line headings under 30 characters
    return [p for p in parts if len(p) > 30]
```

Strategy 5: Recursive Chunking (The LangChain / LlamaIndex Standard)

What if a single paragraph is 5 pages long? Paragraph chunking alone would fail.

Recursive chunking works like an accordion:

1. It tries to split on `\n\n` (paragraphs).
2. If any piece is still larger than `max_size`, it splits that piece on `.` (sentences).
3. If a sentence is still too long, it splits on spaces (words).

```
def chunk_recursive(text, max_size=120, seps=(r"\n\n", r"(?<=[.!?])\s+")):
    text = text.strip()
    if len(text) <= max_size or not seps:
        return [text]

    chunks = []
    # Split using the biggest separator first
    for part in re.split(seps[0], text):
        # Recursively call with the remaining smaller separators
        chunks += chunk_recursive(part, max_size, seps[1:])
    return [c for c in chunks if c]
```

Strategy 6: Semantic Chunking (Embeddings Guide the Knife)

Instead of counting characters, we calculate **cosine similarity between consecutive sentences**. When similarity drops below a threshold, the topic has changed!

```
# 1. Embed every sentence
# 2. Calculate similarity between Sentence(i) and Sentence(i-1)
THRESHOLD = 0.25  # If similarity drops below 0.25, start a new chunk

semantic_chunks = [[sentences[0]]]
for sentence, score in zip(sentences[1:], scores):
    if score < THRESHOLD:
        semantic_chunks.append([])  # Start a brand new chunk!
    semantic_chunks[-1].append(sentence)
```

Strategy 7: Hierarchical Chunking (Parent / Child)

- **Child Chunk:** 1 small sentence (used for high-precision vector search).
- **Parent Chunk:** The whole 1-page section.
- **Mechanism:** Chroma matches the small *child* chunk, but your code fetches and sends the *parent* to the LLM so the model has complete context!

Strategy 10: Modality-Specific Chunking (Tables)

If you blindly cut a table by character count, row 5 loses the column names!

The Solution: Inject column names into every row chunk:

```
table = """Item | Limit | Per
Hotel | 15000 | night
Food | 2000 | day"""

header, *rows = [line.split(" | ") for line in table.split("\n")]
for row in rows:
    # "Item: Hotel. Limit: 15000. Per: night"
    formatted = ". ".join(f"{h}: {v}" for h, v in zip(header, row))
    print(formatted)
```

Step 3: Decision Framework — Which One Should You Pick?

When building a real project, follow this flowchart:

1. **Does the document have clear formatting (headings, blank lines)?**
   □ Use **Paragraph** or **Recursive** (90% of production apps use this).
2. **Is it unstructured continuous text (audio transcripts, WhatsApp chats)?**
   □ Use **Sliding Window** (e.g., 300 tokens with 25% overlap).
3. **Are there financial/numerical tables?**
   □ Use **Modality-Specific** (inject headers into every row).

4. **Is it high-stakes legal/medical where wrong boundaries cause huge harm?**
   □ Use **Semantic** or **AI-Driven**.

Step 4: Putting It Together — Document Chunking Pipeline

Every chunk we create must contain **3 required fields**:

1. `id`: A unique string identifier (e.g., `"leave_policy.txt-1"`). Vector databases require unique keys.
2. `source`: The original file name. Used to cite sources to users.
3. `text`: The actual text payload.

```
from pathlib import Path

def load_documents(folder):
    docs = []
    for path in sorted(Path(folder).glob("*.txt")):
        docs.append({"source": path.name, "text": path.read_text(encoding="utf-8")})
    return docs

def chunk_by_paragraph(text):
    parts = [p.strip() for p in text.split("\n\n")]
    return [p for p in parts if len(p) > 30]

def chunk_documents(docs):
    chunks = []
    for doc in docs:
        for i, text in enumerate(chunk_by_paragraph(doc["text"])):
            chunks.append({
                "id": f"{doc['source']}-{i}",   # Unique ID
                "source": doc["source"],        # Citation file
                "text": text                    # Searchable content
            })
    return chunks

# Execute
docs = load_documents("company_docs")
all_chunks = chunk_documents(docs)

print(f"Loaded {len(docs)} files → Created {len(all_chunks)} chunks.")
print("Sample chunk:", all_chunks[0])
```

□ Key Takeaways

1. **Never embed whole documents:** Multiple topics blur vectors; small chunks preserve specific meaning.
2. **Golden standard size:** 200–400 tokens per chunk.
3. **Always attach metadata:** Without `id` and `source`, you cannot trace where answers came from.
4. **Always inspect chunks:** Before spending money on embeddings, print your chunks in a `pandas.DataFrame` to make sure sentences aren't sliced in half.